Implementing the Chinese Wall Security Model in Security Enhanced Linux

by

Calum Cardownie

Progress Report

University of Strathclyde Glasgow

Department of
Computer and Information Sciences

November, 2025

Contents

1 Aims and Objectives **1**
1.1 Aims . . . 1
1.2 Objectives . . . 1
1.3 Opening Notes . . . 1

2 Summary of Related Work **2**
2.1 Chinese Wall Model Origin . . . 2
2.2 Security Enhanced Linux . . . 3
2.3 Chinese Wall Model in System Security . . . 4

3 Project Specification **5**
3.1 Functional and Non-Functional Requirements . . . 5
3.2 Technology Selection: C Programming Language . . . 5
3.3 Other Language Avenues . . . 6

4 Project Plan including summary of progress **7**
4.1 Phase 1: Developing a static Chinese wall in SELinux . . . 7
4.2 Phase 2: Achieving a dynamic Chinese Wall security model . . . 8
4.3 Phase 3: Analysing the journey to making the Chinese Wall model in SELinux . . . 8

5 Approach to Development **10**
5.1 Methodology . . . 10
5.2 Design . . . 10
5.3 Implementation and Technology ideas . . . 10
5.4 Testing and Evaluation plans . . . 11

A Red Hat System Specs **13**

1 Aims and Objectives

1.1 Aims

The aim of this project is to understand, define and implement the Chinese Wall security model in Security Enhanced Linux (SELinux). As a Chinese Wall in SELinux has never been documented before, this project will document the creation and challenges of implementing the dynamic model in the generally static SELinux environment. As such, the objectives include:

1.2 Objectives

1.1 Creating a suitable environment for SELinux

1.2 Understanding the principles of

- 1.2.1 The Chinese Wall Security model
- 1.2.2 SELinux and its security capabilities

1.3 Defining the Chinese Wall model and its actions in response to `CoI` breaches

1.4 Creating a static implementation of the policy in SELinux

1.5 Creating a daemon that is capable of monitoring subjects and the objects they access

1.6 Adapting the daemon to dynamically update labels to achieve the complete Chinese Wall Security model

1.7 Analyse the difficulties that arose from development and conclude why research in this area has been lacklustre

1.3 Opening Notes

As this implementation is new ground, it must also be considered that a Chinese Wall model in SELinux may be exceptionally challenging. The analysis stage will examine the results regardless of if the Chinese Wall is achieved, but a working implementation still remains a priority of this project. This may mean that this paper's future work might delve into adjacent areas, potentially exploring the viability of a Chinese Wall model in related systems, if the Wall cannot be implemented.

2 Summary of Related Work

2.1 Chinese Wall Model Origin

The Chinese Wall security model - or Brewer and Nash model [2] - is a security model that prevents information from conflicts of interest (`CoI`) groups being spread. As opposed to the Bell LaPadula (BLP) model, which enforces "no read up, no write down" [1], the Chinese Wall model allows for information flow freely until a subject (e.g. a user or process) attempts to read from a `CoI` object (piece of information). Upon this access, the system dynamically revokes their write permissions to other objects in the `CoI` group, creating a "wall" in-between them. Consider the following scenario:

A data server stores information on Banks A and B, and Oil Company A and B. These are in `CoI` groups with their respective counterpart. If a user approaches the dataset, they are initially able to access any object. As far as the system knows, this user is free as they have no prior knowledge of either companies. Suppose the user has read from Bank A, the situation develops - the user now possesses information that, deemed by the Chinese Wall, should not be allowed to reach Bank B, as they are conflicting with Bank A. This is where the wall revokes the write access from the user in regard to Bank B. Now information *cannot* flow from Bank A to B.

- **BN Read Rule**: Subject `S` can read object `O` only if `O` is in the same Company Dataset (`CD`) as an object already read by `S`, or if `O's` (`CD`) is in a different Conflict of Interest (`CoI`) group from all `CD`s of objects already read by `S`.
- **BN Write Rule**: Subject `S` can write to an object `O` only if `S` can read `O` **and** all objects `O'` read by `S` are in the same `CD` as `O`.

The Brewer Nash model provides a mathematical format that is based on the larger Chinese Wall policy, a pillar of compliance in the financial sector. Chinese Walls are a common practice that should theoretically and practically prevent misuse of probable information leakage in banks [5] and other financial management firms. After the Brewer Nash model was communicated, another paper was published once again by Brewer and Nash, but also TY Lin, who presented them with potential improvements on their model [7], leading to the Aggressive Chinese Wall. This idea, trying to disprove the assumption that the conflict of interest is an equivalent relation, has been accepted into the mainstream since it was proposed. This is the type of implementation the 1997 UNIX proof believed in [4], illustrating the possibility of its creation in UNIX. Such proofs in UNIX lead it to be

assumed to theoretically be possible in Linux and SELinux. This proof indicates there is research, but there is a lack of documented attempts to specifically implement it into the UNIX or adjacent kernel.

2.2 Security Enhanced Linux

Security Enhanced Linux implements a Mandatory Access Control (MAC) system, providing a policy-driven security layer extended beyond the classic Discretionary Access Control (DAC) system that UNIX based systems use. DAC is defined such that objects and their privileges are managed by whoever made them, and the root user can access and control anything. DAC in UNIX leverages the `chmod` (change mode) command, allowing read, write and execute access of objects to be edited for the 3 main groups subjects can belong to. While DAC brings about great flexibility, the models' reliance on ownership often leads to uncontrolled privilege propagation. It is known to be difficult to measure the security of the DAC model, and in some cases it is undecidable [3]. MAC is the other model that shares the spotlight with DAC in system security. It utilises, labels, attached to both subjects and objects. Installed policies into the security server govern what labels can interact. Labels are devised in the form `user_u:role_r:type_t:c0` For instance, a base policy may be that an object of type `httpd_t` can only be operated on by a subject of type `httpd_t` and not `sshd_t`. This provides strong mitigation against broken access control damages, as subjects can only access objects that they should and are unable to change the policy controls. For this reason SELinux is adopted by many distributions of Linux, including Red Hat, Fedora, CentOS and even mobile operating system giant Android. SELinux is based off of the Flux Advanced Security Kernel (FLASK) security architecture for flexible nondiscretionary access controls, where the architecture is split into policy enforcement and policy decision-making. SELinux's origin can be attributed to the National Security Agency's Trusted UNIX (TRUSIX) Working Group, working together to publish a Rainbow Book (A series of Computer Security standards and guidelines by the United States Government), and created a second (unpublished) book that included a formal SELinux model and associated evaluation evidence prototype. SELinux was a demonstration that mandatory access control was viable and could be added to the Linux kernel, and so it was merged into the mainline kernel in the 2.6 series of the kernel. Linux was able to adopt this because of its derivation from UNIX, Linux being an open source alternative to the UNIX that was locked down by ATT. Linus Torvalds combined his Linux kernel with the GNU system tools to create GNU/Linux, leading to

what we now know as Linux. This open-source philosophy and flexibility were key factors enabling SELinux's integration into the mainline kernel in the 2.6 series.

2.3 Chinese Wall Model in System Security

Cloud computing is an avenue that has had a successful Chinese Wall model implementation [8]. In this case, there are many Virtual Machines (VMs) running on one system, with the virtual machine monitor, or "hypervisor" acting as the delegating force of the computers' hardware. Each virtual machine is a separate operating system, commonly being completely independent of the other VMs on a machine. Inter-VM attacks can happen if a VM on the system is breached, and can lead to one infected VM spreading to the whole system. This is where the Chinese Wall was created, to eliminate the possibility of inter-VM attacks from competitors. The security model was implemented on the Kernel-base Virtual Machine (KVM) level. Once a VM in a `CoI` group is running on a physical host, the Wall prevents other VMs belonging to the same `CoI` group from being scheduled on the same machine. This effectively eliminates the risk of inter-VM attacks in these systems.

Beyond virtualized environments, the Chinese Wall model has also successfully been implemented in workflow management systems. Workflow Management Systems (WfMS) are intrinsically network-based applications, potentially hosting multiple conflicting sources of information. As such, a secure WfMS must implement security features, specifically targeting information confidentiality as a main point of effort. The Chinese Wall security model can be leveraged to prevent conflict of interest leakage. A Chinese Wall was implemented using role-based access control (RBAC) to enforce, and Java to handle the Chinese Wall logic [6]. The paper mentions that RBAC is unable to implement general cases of the Chinese Wall, as it has issues implementing the BN read rule in dynamic contexts.

3 Project Specification

3.1 Functional and Non-Functional Requirements

Functional Requirements:

- `cwalld` must run as a daemon in the user-space of the computer until terminated
- `cwalld` must monitor read and write actions on all subjects on the system
- `cwalld` must identify `CoI` triggers to act upon
- `cwalld` must create a policy module that will prevent the `CoI` from being breached
- `cwalld` must install the policy on the machine
- `cwalld` will write to a log file documenting policies it installs

Non-Functional Requirements:

- `cwalld` will only be controllable by the admin
- `cwalld` will handle memory in a thread-safe and careful manner such to avoid concurrency issues and memory leaks
- `cwalld` must not significantly impact performance of the system

3.2 Technology Selection: C Programming Language

The Chinese Wall security daemon will be implemented in `C`, elected for its suitability for kernel and system-level programming. The main reasons for this choice are documented below

- **System Program Status**: `C` has been the *de-facto* standard language for operating systems programming since its origin in the 1970s, with major kernels including Linux, Windows NT and freeBSD being predominantly written in `C`.
- **Low-level Precision**: `C` is a language that allows the developer incredible freedom, including the potential to manipulate memory directly, and interact with core parts of the system. The daemon requires direct interaction with system calls and process management, which `C` provides natively.

- **Performance**: `C` is a compiled language with minute overhead, ensuring that the daemon introduces minimal latency to the system.
- **Familiarity**: I have prior experience with `C`, developing several low level projects with it, courtesy of CS210.

The use of `C` introduces some challenges however, listed below

- **Memory Management**: Given `C` grants the developer power to manipulate memory, it must be managed with disciplined design practices. If memory is not managed perfectly it can lead to memory leaks and buffer overflows, that will build up and completely eliminate the daemon's viability.
- **Concurrency**: System daemon tasks inherently involve concurrency, as such practices such as mutex locks and semaphores must be considered to prevent race conditions, and the daemon must follow thread-safe design patterns.

3.3 Other Language Avenues

Other languages that were considered include:

- **Rust**: `Rust` is hailed for its type safety and security, while also compiling to near C-level speed, making it a great choice for daemon development, however, the steep learning curve makes it a difficult choice.
- **Go**: `Go` is a modern, slightly higher level alternative to `C`, with excellent concurrent integration with its Goroutines, but the garbage collection introduces latency and an indirect method of interaction with SELinux libraries leave it in the shadow of `C`.
- **Python**: While recent developments with `Python 3.14` have removed the global interpreter lock that prevented true multithreading, it remains an interpreted language that cannot match the speed and minimal overhead of compiled languages such as `C`.

4 Project Plan including summary of progress

The project has 3 main phases.

- **Phase 1**. Developing a static Chinese Wall in SELinux
- **Phase 2**. Achieving a dynamic Chinese Wall security model
- **Phase 3**. Analysing the journey of the project and concluding why it has been void of attention

The setup for the first phase involves setting up an environment that can run SELinux. I have chosen Red Hat as the Linux distribution for this end. Red Hat Enterprise Linux (RHEL) is a popular SELinux distribution that other distributions like Fedora and CentOS are forks and inspired by respectively. The plan outlines using a combination of Kernal-based Virtual Machine (KVM) and Quick Emulator (QEMU) to run the VM. Virt manager, created by the same company as Red Hat, IBM, was chosen to act as an interface to run and interact with the VM. The details of the VM can be found in figure: 2 in the appendix.

4.1 Phase 1: Developing a static Chinese wall in SELinux

The first phase is based on researching and understanding the capabilities of SELinux, and the completion of a static Chinese Wall. The plan for this phase involves scoping out the capabilities of SELinux's security system. So far, a static implementation of a Chinese Wall has been the priority. This involves an object, able to be written and read by subject A, but not subject B, whilst the subjects having the same access everywhere else in the system. This must be implemented with SELinux security labels for properly enforced security. In pursuit of the static wall and pushing the limits of SELinux, I have learnt many things about SELinux systems, including:

- Familiarisation with SELinux libraries (SETroubleshoot, Policycoreutils, SETools-console)
- Identification and modification of SELinux labels and object access
- Creating and confining a simple daemon within an SELinux context
- Creating and installing SELinux type enforcement files

At this point, a base static wall has been completed. I began by attempting to enforce the wall on a user basis, preventing a real Linux user from accessing objects they were in conflict with. It is very

difficult to adapt the type of specifically the user, using a process rather than a standard system running subject. As such, the Chinese Wall may only be able to enforce its protection on a process level and not a system user level.

4.2 Phase 2: Achieving a dynamic Chinese Wall security model

The plan for the second phase primarily involves creation of the Chinese Wall daemon, written in `C`. This daemon will survey the system and modify security modules, designating the denial to the security server. This will be the longest phase, including daemon development, along with testing. Phase 2 will also potentially venture into Kernel level engineering, as a Chinese Wall will require atomic operations to deny subjects based on their history, and installing policy modules is a system-wide operation that will not be able to done freely by a daemon.

4.3 Phase 3: Analysing the journey to making the Chinese Wall model in SELinux

The final phase is simply analysing challenges that arose from creation of the wall. The main challenge so far has been a lack of documentation. While the Red Hat docs are well-written and detailed, they only describe operations that a standard system admin would need, rather than anything more complex like custom policy modules. As such, finding accurate documentation is a challenge for this project, as SELinux has its own policy module language (.te) with its own syntax and logic system.

A Gantt chart was constructed to depict the project timeline:

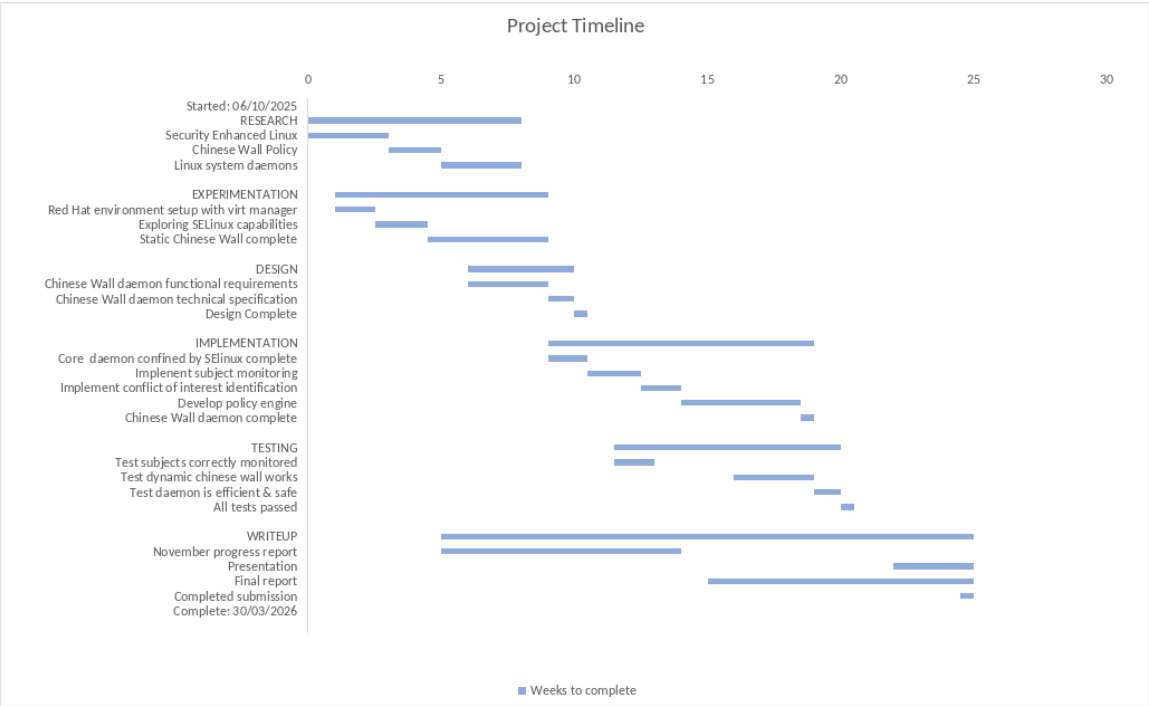

Figure 1: Project Gantt chart

5 Approach to Development

5.1 Methodology

As mentioned earlier, the project has 3 phases. Each phase will refer to a different style of development methodology. The first phase is based on researching and understanding the capabilities of SELinux, and the completion of a static Chinese Wall. This phase is agile, as the needs of the phase will constantly be revisited and analysed. It will allow me to alter the functional and non-functional requirements of the daemon as I scope the SELinux environment out. The second phase will oversee the development process of the Chinese Wall daemon `cwalld`. This phase will follow the waterfall principle. By this phase, the daemon will have a set of defined requirements, and all focus will be on developing the daemon to meet those ends. Regardless of phase 2's success, there will be enough time to approach phase 3. Phase 3 will delve into the reasoning behind the lack of Chinese Wall development in SELinux.

5.2 Design

The design of the Chinese Wall security policy is such that once a subject reads from an object that is related to a conflict of interest group, their write access to other objects within that conflict of interest is revoked. This prevents information from one object to reach a competitor.

SELinux is designed such that only subjects can only access objects if the policy allows its labels to. In standard SELinux, these labels are all static and are only changed by the user under unique circumstances. Allowing dynamic policies would involve having type enforcement (.te) files edited and installed live based on the labels of the subjects. This is where the Chinese Wall daemon is required.

5.3 Implementation and Technology ideas

The base language C comes completely fresh. As such, there will be usages of common C libraries, ranging from the standard library, daemon infrastructure, SELinux core, process monitoring and system operations. The project will likely also involve kernel programming, which is notoriously difficult. This will be evaluated once that stage is reached.

5.4 Testing and Evaluation plans

Testing depends on the amount of time that remains once the daemon is complete, as it being operational will be a big bottleneck for the project. Once ready the daemon will be tested in efficiency, robustness security. The daemon's resource usage will be measured and compared to other system daemons. Measuring robustness and security, the daemon will be tested to operate on a huge amount of subjects and objects at once, and it will be noted if it was able to efficiently execute its operations without issue. The report will be evaluated based on the completion of the daemon, and it's ability to enforce the Chinese Wall security policy.

References

[1] D. E. Bell and L. J. Lapadula. Secure computer systems: Mathematical foundations. Number: MTR2547VOL1.

[2] D. Brewer and M. Nash. The chinese wall security policy. In *Proceedings. 1989 IEEE Symposium on Security and Privacy*, pages 206–214. IEEE Comput. Soc. Press.

[3] Y. Fan, Z. Han, J. Liu, and Y. Zhao. A mandatory access control model with enhanced flexibility. In *2009 International Conference on Multimedia Information Networking and Security*, volume 1, pages 120–124. ISSN: 2162-8998.

[4] S. N. Foley. Building chinese walls in standard unixTM. 16(6):551–563.

[5] L. Herzel and D. E. Colling. The chinese wall and conflict of interest in banks. 34(1):73–115. Publisher: American Bar Association.

[6] Y.-C. Hsiao and G.-H. Hwang. Implementing the chinese wall security model in workflow management systems. In *International Symposium on Parallel and Distributed Processing with Applications*, pages 574–581. ISSN: 2158-9208.

[7] T. Y. Lin. Chinese wall security policy-an aggressive model. pages 282,283,284,285,286,287,288,289–282,283,284,285,286,287,288,289. IEEE Computer Society.

[8] T.-H. Tsai, Y.-C. Chen, H.-C. Huang, P.-M. Huang, and K.-S. Chou. A practical chinese wall security model in cloud computing. In *2011 13th Asia-Pacific Network Operations and Management Symposium*, pages 1–4.

A Red Hat System Specs

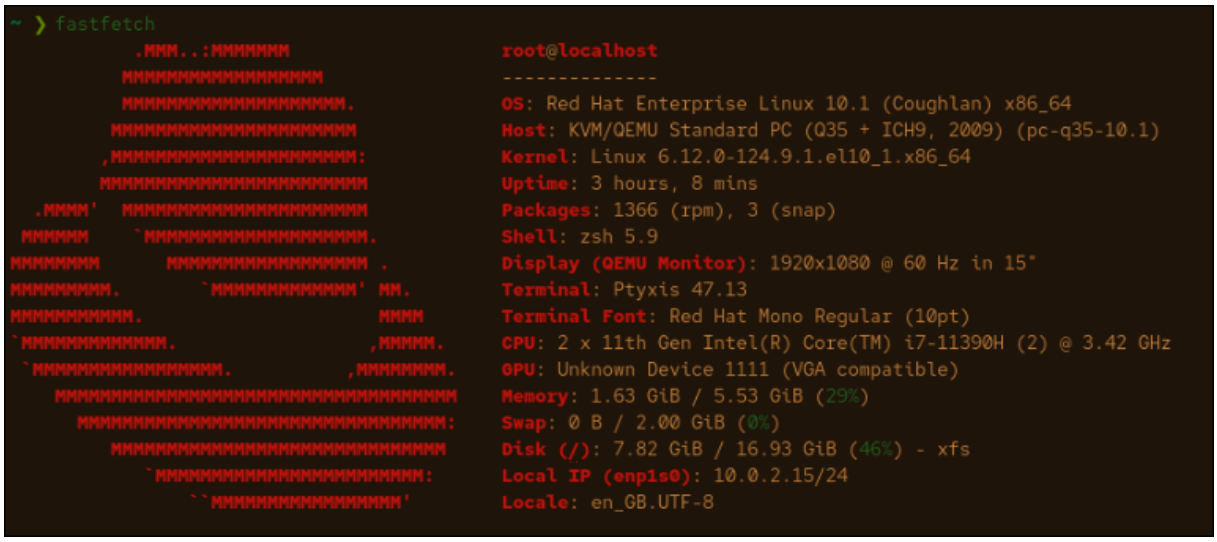

Figure 2: Red Hat system specs